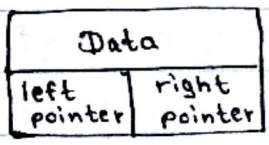
No: Date: / /

234067X Lab Activity 01 (BTS)

Ramage G.D.K.N.

(01) Data
Left pointer
Right pointer



(02) To maintain the correct links between nodes after insertion or deletion. (ensure changes aren't lost update tree after any change.)

(03) It improves search by dividing data values below root go to left while above values go to right sub tree so, we skip half of the tree at each step.

(04)

Recursive	Iterative
Function calls itself	uses loop
Uses call stack	No extra stack
short, clean code	longer code
slow	fast

(05) We use resursion in BST because each subtree is itself a BST, and recursion naturally handles this structure.

```
Node insert (Node root, int key) {
    if (root == NULL)
        return new Node(key);

    if (key < root.data)
        root.left = insert(root.left, key);
    else if (key > root.data)
        root.right = insert(root.right, key);

    return root;
}
```

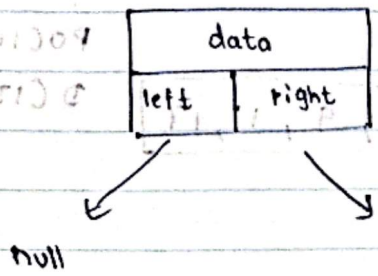
06 Base condition stops recursion

```
if (root == null)
    return root;
```

07 Infinite recursion occors

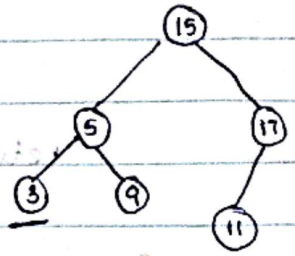
Program crashes. (Will cause stack overflow error.)

08 A leaf node has no children.



```
if (node.left == null && node.right == null) {
    System.out.println("Leaf node");
}
```

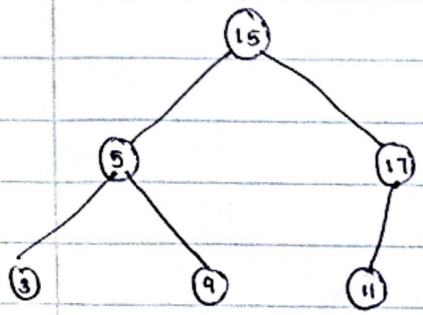
(09) The smallest value in always in the leftmost node.




```
int findMin(Node root) {
    ~~if (root == null)~~
    while (root.left != null) {
        root = root.left;
    }
    return root.data;
}
```

(10) Inorder traversal gives sorted output because it visits nodes in the order left subtree, root and right subtree, which follows th BST property

Left → Root → Right



PO(5) PO(3) D(3) D(9)
D(15) D(5) D(5) PO(11)
PO(17) PO(9) PO(9)
PO(17) PO(17)

PO(11) D(11)
D(17) D(17)

3, 5, 9, 11, 17